FIT2099 Assignment 2 Design Documentation

Design Improvement on Assignment 1:

A. Create Utils final class and add utility function for generating random numbers

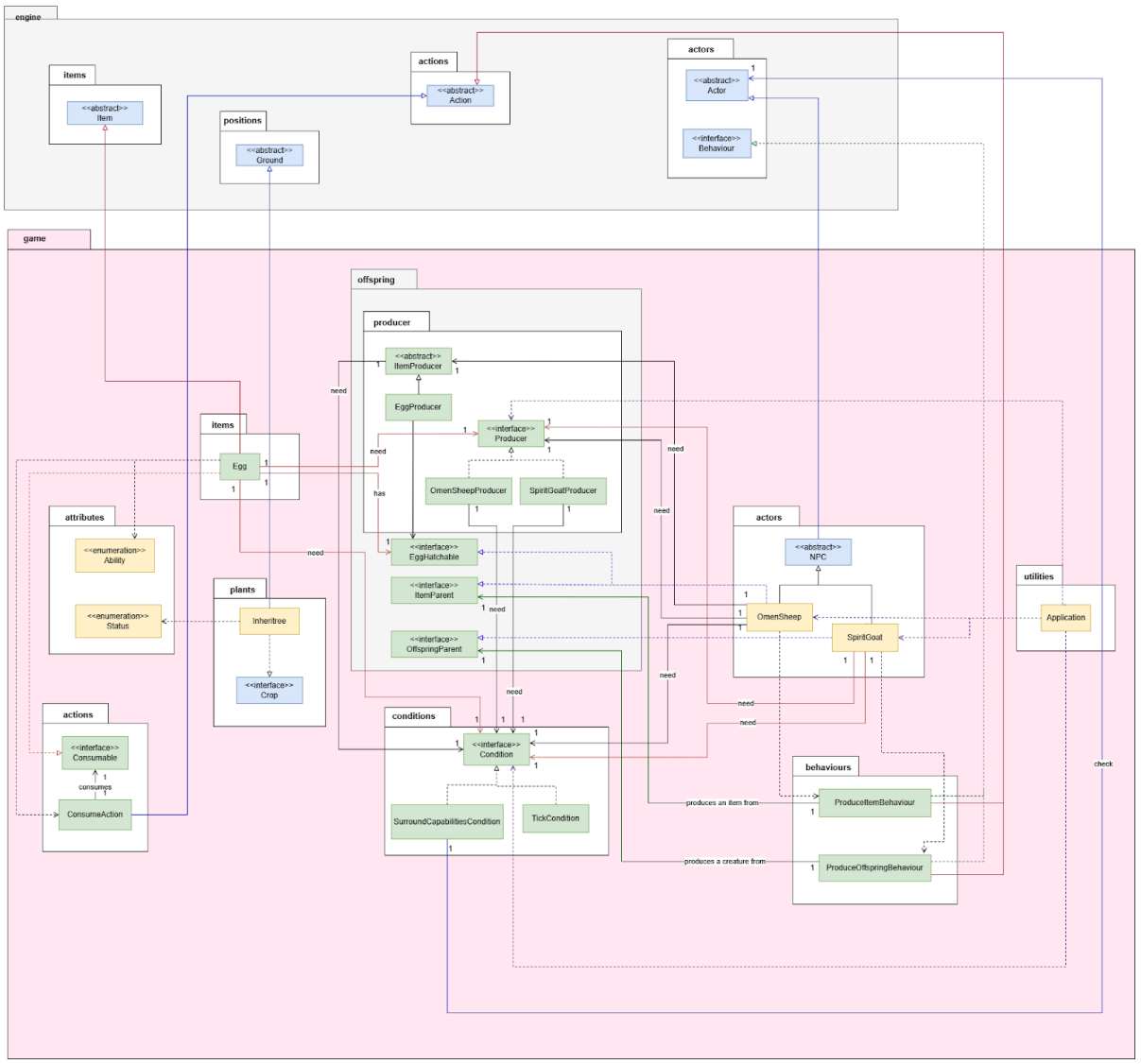
Pros	**Cons**
Promote code reusability	Unnecessary complexity if only involve simple logic
Improve maintainability because changes of implementation logic can be done in one place	

Design Assumption:

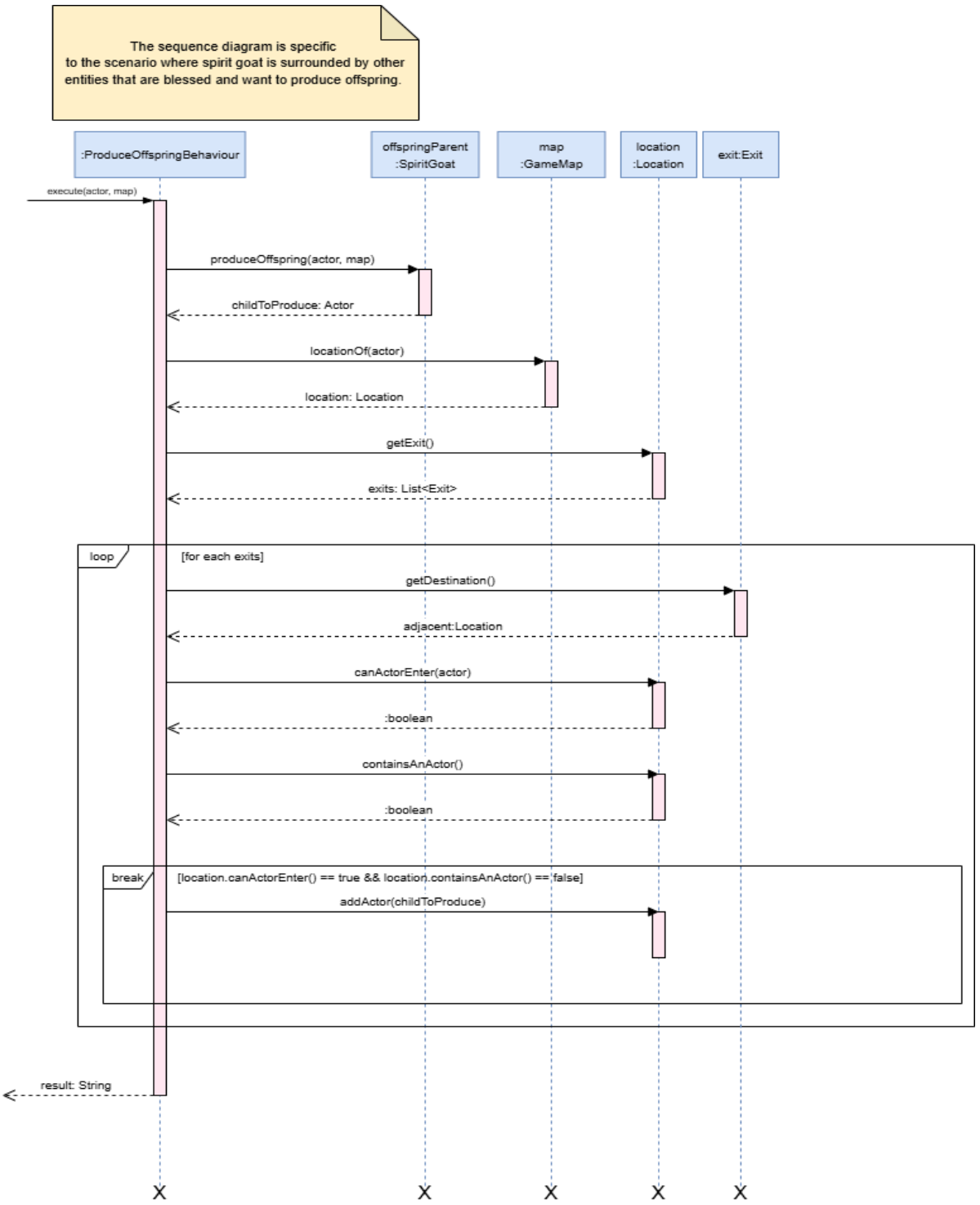
1. One egg-laying actor can only reproduce one item.
2. Items or creatures produced will inherit the parent's reproduction and hatching condition for now.

REQ 1

UML Diagram



Sequence Diagram



Design Rationale

Design 1

Create OffspringParent & ItemParent interface to represent targets that can produce offspring (e.g. SpiritGoat) and to represent targets that can produce item (e.g. OmenSheep).

Pros	Cons
Each interface focuses on its own reproduction type and only on production capability, separating from other capabilities. (SRP)	More classes be created
Implementers only need to override methods required for reproduction, avoiding unnecessary methods. (ISP)	
New OffSpringParent target can simply implement this interface without modifying other classes. (OCP)	

Design 2

Create ProduceOffSpringBehaviour class to personalise reproduction action taken by a target that has the capability to produce offspring, (e.g. SpiritGoat).

Create ProduceItemBehaviour class to personalise reproduction action taken by a target that has the capability to produce an item, (e.g. OmenSheep).

These behavioural classes extend Action class and implements Behaviour interface to override execution, aiming to allow targeted creatures to reproduce when condition is met.

Pros	Cons
Encapsulate reproducing behaviour based on its own reproduction type, separating from other behaviour. (SRP)	Requires extra attention on ensuring the reproducing logic applied to every OffspringParent/ItemParent implementers.
NPC classes can add these behaviours without knowing its implementation details, promoting information hiding.	Extending Action and implementing Behaviour can violate SRP

Design 4

Create Egg class to represent egg laid by a creature.

This class extends Item abstract class to override tick execution to hatch an egg into a target creature when hatching condition is met; implements Consumable interface to invoke its personalized consuming effect (all eggs can be consumed).

Pros	Cons
Less class needed	Less flexible for specific egg has own implementation logic
New egg type introduced only requires passing its name, parent, hatching condition and hatch target as argument, promoting scalability and maintainability.	
Safely substitutable for Item because it overrides behavior without changing expected contracts. (LSP)	

Design 5

Create a Consumable interface to represent an edible entity, (e.g. Egg class).

Pros	Cons
Ensure consumable target implements consume method defined in this interface to personalise own consuming effect, enforce its interchangeability without breaking functionality. (LSP)	More class be created
Focuses on handling consuming effect by avoiding unnecessary methods. (ISP)	
New consumable entities can simply implement this interface without modifying other classes. (OCP)	

Design 6

Create a ConsumeAction class to define consuming behaviour when an egg is in farmer inventory and farmer chooses to consume the egg.

This class extends from Actions abstract class to override execution and has association with Consumable interface.

Pros	Cons
Encapsulate consuming functionality only, separating from other actions. (SRP)	Tight coupling to Consumable interface, forcing all consumption logic into a single consume(). These risks violate SRP and create a connascence of timing, where multiple side effects must occur in a specific order.
Simply by adding a ConsumeAction instance to Egg class, farmer can consume egg without knowing its implementation details, promoting information hiding.	
Depends on abstraction – Action abstract class and Consumable interface but not concrete classes (Egg), avoid downcasting and using instanceof(). (DIP)	
Extending from Action allows consuming logic to be centralized to one method (execute()), allowing consistent consumption when actors choose to consume a consumable. Changes to consuming logic only require updating this method, reducing redundancy. (DRY)	

Design 7

Create CONSUME Ability enum value to indicate entity with consume entity ability (e.g. add to Player).

Pros	Cons
Avoid using instanceof to check if an entity belongs to a particular class.	Allows easy ‘extend’ ability, status, and attribute, but handling them usually requires modifying existing logic, such as allowableActions(). (violate OCP).

Design 8

Create an EggHatchable interface to let egg-laying actors personalise their own egg consumption effect.

Pros	Cons
Avoid using instanceof to check if an egg belongs to a particular egg type by their Egg instance name, avoid hardcoding logic. (OCP)	Forces all eggs to have an effect when being consumed even if not applicable – may lead to empty logic. (violate ISP)
Focuses on handling consuming effect by avoiding unnecessary methods. (ISP)	
Supports polymorphism, Egg can trigger consume effect through EggHatchable without knowing the actual actor type (DIP)	

Design 9

Create a Producer interface to indicate a class acting as the producer of a reproducible npc type, (e.g. SpiritGoatProducer and OmenSheepProducer).

Pros	Cons
Implementers only need to override methods required for new creation, avoiding unnecessary methods. (ISP)	May require modification when more and complex reproduction conditions are introduced for a single reproduction behaviour in the future. (violate OCP)
New creature's producer can simply implement this interface without modifying other classes. (OCP)	

Design 10

Create an ItemProducer abstract class indicating a class acting as the producer of an Item, (e.g. EggProducer class).

This class has association with Condition and Producer interface to store the condition to produce a specific item and the target creature to produce from the item.

Pros	Cons
Depends on abstraction - Condition and Producer interface but not concrete classes, avoid downcasting and using instanceof. (DIP)	Tighter coupling for inheritance
EggProducer inherits common logic without redefining it multiple times, avoiding code duplication. (DRY)	

Design 11

Create SpiritGoatProducer & OmenSheepProducer classes to create a new spirit goat/omen sheep instance based on reproduction condition.

These classes implement Producer interface to override the creation of each npc.

Pros	Cons
Encapsulate creation logic for each reproducible npc into its own class, avoiding exposure of internal reproduction condition. (SRP)	Could lead to overengineering if each NPC needs its own producer with only minor differences.
Can be used polymorphically in any generic producer context. (LSP)	

Design 12

Create a Condition interface to represent a general condition that can be satisfied either by an Actor or at a Location. This interface is implemented by classes that define logic to determine whether certain behaviours (eg. hatching) should occur.

Pros	Cons
New condition types (eg. attribute-based) can be added by creating new classes without modifying existing ones (OCP)	More classes needed for specific condition classes.

Provides a default method for Location, enabling optional override to support both actor-based and location-based condition logic without forcing implementation (ISP)	

Design 13

Create a SurroundCapabilitiesCondition class to check if any surrounding tile contains an entity with a specified capability by implementing the Condition interface to support both actor-based and location-based condition checking.

Pros	Cons
Combine all entity types into one condition check, reducing the need to separate condition classes for each (SRP)	More classes needed for specific condition classes.
Overloads both satisfies(Actor, GameMap) and satisfies(Location) to support multiple use cases Eg. egg hatching via location or ability triggers via actor (OCP)	
Uses a shared helper() method to avoid repeating capability-checking logic across both overloads (DRY)	

Design 14

Create a TickCondition class to represent a condition that is satisfied based on turn count by implementing the Condition interface and increasing a tick counter by a fixed rate every time the condition is checked, returning true when the tick count reaches a specified threshold.

Pros	Cons
Allows reusable turns-based condition checking for different purposes (SRP)	Complex to implement
Tick rate and threshold can be passed as parameters to customise timing without modifying the class (OCP)	

Supports both actor-based and location-based contexts through two overridden methods	

SOLID, DRY and connascence principle

Every class only focuses on one responsibility. OffspringParent interface focuses on common reproduce offspring logic; ItemParent interface focuses on common laying egg logic; ProduceOffspringBehaviour and ProduceItemBehaviour classes encapsulate reproduction behaviour based on its own reproduction type; Egg class focuses on hatching into a creature; Consumable interface defines consume effect; EggHatchable interface defines consume effect of egg; ConsumeAction class focuses on executing consume logic when action is invoked; Producer interface defines creation logic; ItemProducer abstract class handle common logic for creating new item type; EggProducer inherits ItemProducer abstract class focuses on producing an egg; SpiritGoatProducer and OmenSheepProducer classes encapsulate the creation of each npc type based on its condition; Condition interface defines condition need to satisfy before perform certain behaviours; SurroundCapabilitiesCondition class encapsulate the validation check of surrounding on the map; TickCondition class encapsulate the comparison operation between current number of turn and its threshold. These designs show the **Single Responsibility Principle (SRP)** is achieved.

When a new reproducible target is introduced, we can simply let the target class implements its parent interface based on its reproduction type, (i.e. OffspringParent/ItemParent interface), without modifying existing classes. ItemProducer abstract class also encourages easy extensibility. Besides the EggProducer subclass, new item types which can be produced may be introduced in the future, and it can be easily extended from this abstract class. Considering the case where more conditions may be introduced in the future, these conditions can also be easily extended by using Condition interface without modifying other condition subclasses. These designs certainly avoid using if-else statements or instanceof() to check if an entity is an instance with reproduction ability or consumable, supporting the **Open-Closed Principle (OCP)** by being open to extension but close to modification.

Creating OffspringParent interface ensures target produce offspring implements canProduce() and produceOffspring() defined in this interface to personalise own reproduction condition and behaviour in the valley, which further contribute to enforcing its consistency and interchangeability without breaking functionality. This concept also applies to ItemParent and Consumable interface, where every consumable target must implement consume() to personalise its own consumption logic. Besides, EggProducer is a subclass of ItemProducer abstract class. The abstract methods, turnIntoItem() defined in abstract class ensures EggProducer implements the same method and expected behaviour. In other words, wherever itemProducer instance is expected, EggProducer can be substituted without breaking the behaviour of the program. It maintains the contract of its parent class. The same concept applies to Egg class, which ensures safe substitution when an Item instance is expected. These designs prevent inconsistent logic, which achieve **Liskov Substitution Principle (LSP).**

OffspringParent interface has only produceOffspring() and canProduce() methods, focus on handling offspring behaviour; ItemParent interface has only reproduce() and canProduce() methods, focus on handling producing item behaviour such as laying egg; Consumable interface has only one consume() method and Condition interface has only two satisfies() methods, each define the condition needed by Actor/Item to satisfy to perform certain behaviour. These designs avoid unnecessary methods, keeping them focused and minimal. Entity instances without reproduction and consumable capabilities do not need to implement these interfaces, preventing unused methods or forced dependencies. This means that **Interface Segregation Principle (ISP**) is followed. Nevertheless, extra concern is needed where our current design forces all eggs to have an effect in valley when being consumed even if not applicable (I,e, egg being consumed and nothing happens) – may lead to empty logic and violate ISP.

ProduceOffspringBehaviour and ProduceItemBehaviour classes depends on abstraction – extends from Action abstract class, implements Behaviour interface and has association to OffspringParent/ItemParent interfaces rather than concrete classes like SpiritGoat. Additionally, ConsumeAction class depends on Action abstract class and has association to Consumable interface rather than concrete classes like Egg class. This allows adding new reproducible or consumable targets easily without causing heavy dependency on lower modules, avoid downcasting and the violation of Open-Closed Principle (OCP) as well. The same concept applies to the ItemProducer abstract class, which depends on Condition and Producer interface but not concrete classes like Egg class. This means that the Dependency **Inversion Principle (DIP)** is followed.

TickCondition and SurroundCapabilitiesCondition can be reused over every part of code when needed to check Condition for tick or surrounding just by defining the specific tick threshold and capabilities to check, especially for reproducing condition. In addition, ConsumeAction class extending from Action allows consuming logic to be centralized to one method (execute()), changes to consuming logic only require updating this method, reducing redundancy in defining the same action in all classes. For classes which implement the Consumable interface, we can easily allow the target to be consumed by farmers by invoking the ConsumeAction instance in allowableActions(), avoiding repeating the same logic across all classes. Moreover, SurroundCapabilitiesCondition class consists of a `helper()` which is reusable across methods, to avoid repeating capability-checking logic across both overloads. **(DRY)**

By encapsulating the surrounding capabilities checker into a single helper() method in SurroundCapabilitiesCondition class, we can avoid duplicating the same logic across different methods since both satisfies() methods use the same logic. This reduces **connascence of algorithm** as changes on the checker only require updating helper() instead of both satisfy() methods. This concept also applied to SpiritGoat class. In Assignment 1, cure() directly modifies the TIMER attribute value. Now, the countdown reset logic is abstracted into another private method called resetCountdoenTimer(), so the external class doesn't need to know how the timer resets.